

Подготовка к защите проекта Stylish

Разбор проекта для уверенной защиты перед учителем.

1. Что это за проект одной фразой

Stylish - веб-приложение <<умный гардероб>>:

- пользователь регистрируется и входит;
- создаёт профиль;
- загружает фото себя и вещи (футболка, джинсы и т.д.);
- генерирует образ (look) - <<как я выгляжу в этих вещах>>;
- всё работает через микросервисную архитектуру в Docker.

Идея для защиты: <<Я сделал не монолит, а разделил систему на независимые сервисы - авторизация, бизнес-логика гардероба и ML-генерация - чтобы каждый можно было масштабировать и развивать отдельно>>.

2. Общая архитектура

Пользователь (браузер)	
------------------------	--

Стек технологий

Слой	Технологии
Backend	Java 17, Spring Boot 4.0.3, Maven (multi-module)
Frontend	React + TypeScript + Vite
БД	PostgreSQL (основные данные), MongoDB (аудит/активность)
Файлы	MinIO (S3-совместимое хранилище)
Инфра	Docker Compose

Безопасность	Spring Security + JWT (HS256)
Документация API	Swagger (springdoc-openapi)

Структура папок

MUTANT4/

— pom.xml

Parent POM (pom.xml) - корень Maven-проекта. Объединяет три модуля и задаёт общие версии (Java 17, Spring Boot 4.0.3).

3. Три микросервиса - кто за что отвечает

auth-service (порт 8081) - <<охранник и выдача пропусков>>

Задача: всё, что связано с пользователями и токенами.

Эндпоинт	Что делает
POST /auth/register	Регистрация (email + пароль)
POST /auth/login	Вход -> выдаёт access + refresh токены
POST /auth/refresh	Обновляет пару токенов
POST /auth/logout	Отзывает refresh-токен
GET /auth/me	Данные текущего пользователя (нужен JWT)
GET /auth/events/me	История входов из MongoDB

Где хранит данные:

- PostgreSQL - таблицы users, refresh_tokens
- MongoDB (authdb) - коллекция auth_events (аудит: кто когда логинился)

wardrobe-service (порт 8082) - <<основной бизнес>>

Задача: профиль, вещи, фото, образы, файлы.

Эндпоинт	Что делает
POST /profiles	Создать профиль

GET /profiles/me	Мой профиль
POST /items	Загрузить вещь (multipart: JSON + картинка)
GET /items	Список вещей
POST /user-photos	Загрузить своё фото
POST /looks/generate	Сгенерировать образ
GET /files/{key}	Скачать файл из MinIO
GET /activities/me	Лента активности из MongoDB

Где хранит данные:

- PostgreSQL - profiles, items, user_photos, looks, look_items
- MongoDB (wardrobedb) - activity_events
- MinIO - сами картинки (файлы)

generation-service (порт 8083) - <<ML-заглушка>>

Задача: принять запрос на генерацию и вернуть PNG-картинку.

Сейчас это stub (заглушка): StubGenerationService рисует простую картинку с текстом <<Stylish (stub)>> через Java AWT. В реальном проекте сюда подключили бы нейросеть.

Зачем отдельный сервис: ML тяжёлый, его можно масштабировать отдельно, менять модель, не трогая wardrobe-service.

4. Базы данных - почему две и зачем каждая

PostgreSQL - <<истина>> о данных

Реляционная БД для структурированных данных с связями:

users (auth)	profiles (wardrobe)
--------------	---------------------



Flyway - инструмент миграций. При старте сервиса SQL-скрипты из db/migration/V1__init.sql автоматически создают таблицы. Это версионирование схемы БД.

Что сказать учителю: <<PostgreSQL - для транзакционных данных с foreign key. Flyway гарантирует, что схема одинакова у всех разработчиков и на проде>>.

MongoDB - <<журналы и ленты>>

Document-oriented БД для логов, где не нужны жёсткие связи:

База	Коллекция	Что пишется
authdb	auth_events	REGISTER, LOGIN_SUCCESS, LOGIN_FAILED, RE
wardrobedb	activity_events	ITEM_CREATED, LOOK_GENERATED, USER_PHO

Почему Mongo, а не Postgres:

- много записей <<append-only>> (только добавляем);
- гибкая структура документа;
- не блокируем основной HTTP-запрос - пишем асинхронно (@Async).

MinIO - <<файловое хранилище>>

S3-совместимое хранилище для картинок. Бакет stylish приватный - снаружи файлы не доступны. Скачивание только через GET /files/{key} в wardrobe-service.

Ключи файлов:

- items/{profileId}/{uuid}.jpg - вещи
- user-photos/{profileId}/{uuid}.jpg - фото пользователя
- looks/{profileId}/{uuid}.png - результат генерации

5. Безопасность - как работает JWT

Два типа токенов

Токен	Срок жизни	Где хранится	Для чего
Access	15 минут	sessionStorage (фронт)	Каждый API-запрос: Authorization: Bearer
Refresh	30 дней	localStorage (фронт)	Обновить access, когда тот протух

Как это работает по шагам

1. Пользователь логинится -> POST /auth/login

2. auth-se

Важные детали для защиты

1. auth-service - issuer (выпускает токены)

- Алгоритм HS256, секрет из env JWT_ACCESS_SECRET

- Claims: sub = userId, email, iat, exp
 - Пароли - BCrypt (не храним в открытом виде)
2. wardrobe-service - resource server (проверяет токены)
- Тот же секрет -> проверяет JWT локально, без запроса в auth-service
 - currentUser.userId() достаёт UUID из sub
3. Refresh с ротацией
- Формат: {uuid}.{secret}
 - В БД хранится только BCrypt-хэш secret
 - При refresh старый токен помечается revoked_at, выдаётся новая пара
4. Изоляция данных (ownership)
- Все запросы к items/looks фильтруются по profile_id текущего пользователя
 - findByIdAndProfile_Id(...) - чужие данные недостижимы даже если знаешь UUID
-

6. Главные сценарии - что происходит <<под капотом>>

Сценарий 1: Регистрация и вход

1. POST /auth/register -> email нормализуется, пароль хэшируется BCrypt

2. User co

Сценарий 2: Загрузка вещи

1. POST /items (multipart: metadata JSON + file)

2. wardro

Сценарий 3: Генерация образа (look)

1. POST /looks/generate { userPhotoId, itemIds[], name? }

2. Провер

7. Frontend - как устроен

Страницы

- /auth - логин/регистрация
- /app - дашборд (профиль, фото, вещи, образы)

Хранение токенов

- accessToken -> sessionStorage (пропадает при закрытии вкладки)
- refreshToken -> localStorage (переживает перезагрузку)

Авто-refresh

При перезагрузке страницы, если access нет, но refresh есть - фронт тихо вызывает POST /auth/refresh и восстанавливает сессию (router.tsx).

HTTP-клиент (http.ts)

- Все запросы идут с Authorization: Bearer ...
- Если пришёл 401 - автоматически пробует refresh и повторяет запрос
- Для загрузки файлов - отдельный apiFetchForm (multipart)

UI-компоненты дашборда

- ProfileCard - создать/показать профиль
- UserPhotoCard - загрузить фото себя
- ItemsCard - CRUD вещей
- LooksCard - генерация и просмотр образов

8. Технологии из задания - где что лежит

Требование	Где реализовано	Зачем
Docker	docker-compose.yml	8 контейнеров, healthchecks, depends_on
Swagger	Каждый сервис, /swagger-ui.html	Документация и тест API в браузере
Async	AuthEventService, ActivityService	Не блокировать HTTP при записи в Mongo
Cache	Caffeine в auth + wardrobe	Кэш user-by-email и profile-by-userId
Testing	*ServiceTest.java	Unit-тесты JUnit 5 + Mockito
File upload	POST /items, POST /user-photos	Multipart -> MinIO

File download	GET /files/{key}	Стрим из MinIO
Spring Security	SecurityConfig.java	Stateless, JWT, BCrypt
JWT	auth выпускает, wardrobe проверяет	Access 15 мин, refresh 30 дней
MongoDB	auth_events, activity_events	Аудит и лента активности
Flyway	V1__init.sql	Миграции Postgres

9. Как запустить и показать учителю

```
docker compose up --build
```

URL	Что показать
http://localhost:3000	Фронт: регистрация -> дашборд
http://localhost:8081/swagger-ui.html	Auth API
http://localhost:8082/swagger-ui.html	Wardrobe API
http://localhost:8083/swagger-ui.html	Generation API
http://localhost:9001	MinIO console (stylish / stylish123)

Демо через Swagger:

1. POST /auth/register -> POST /auth/login -> скопировать accessToken

2. Authori

10. Что говорить на защите - готовые формулировки

Вступление (30 секунд)

Проект Stylish - микросервисное приложение для виртуального гардероба. Пользователь загружает фото и вещи, система генерирует образ. Архитектура: три Spring Boot сервиса, React-фронт, PostgreSQL для данных, MongoDB для аудита, MinIO для файлов. Всё поднимается через Docker Compose.

Почему микросервисы

Auth, wardrobe и generation - разные зоны ответственности. Auth можно масштабировать отдельно при пиках логинов. Generation - тяжёлый ML, его можно вынести на GPU-сервер. Wardrobe - основной CRUD и файлы.

Почему JWT, а не сессии

Stateless: wardrobe-service проверяет токен локально, без обращения к auth на каждый запрос. Это быстрее и проще масштабировать. Refresh-токены с ротацией дают безопасность при компрометации.

Почему Async для Mongo

Запись аудита не должна замедлять login или создание item. @Async отправляет запись в отдельный thread pool, HTTP-ответ уходит сразу.

Почему Cache

Profile читается на каждом запросе в wardrobe. Без кэша - лишние SELECT в Postgres. Caffeine in-memory с TTL 10 минут.

Про generation-service

Сейчас stub - рисует placeholder PNG. Архитектура готова: wardrobe вызывает generation через HTTP, можно заменить на реальную ML-модель без изменения API.

11. Вопросы учителя - и ответы

Q: Почему два типа БД?

A: PostgreSQL
append-only

Q: Как wardrobe узнаёт, кто пользователь?

A: JWT в
ищем про

Q: Может ли пользователь А увидеть вещи пользователя В?

A: Нет. В
JWT. Чуж

Q: Где хранятся пароли?

A: Только

Q: Что если access-токен украли?

A: Живёт
(ротация)

Q: Зачем MinIO, а не файлы на диске?

A: S3-сов
доступ то

Q: Что такое Flyway?

A: Версия
старте. О

Q: Есть ли тесты?

A: Unit-тесты
JUnit 5 + Mockito

Q: Почему generation - отдельный сервис?

A: ML-модель
реализация

12. Слои внутри каждого Spring-сервиса

Типичная структура (на примере wardrobe):

Controller → принимает HTTP, валидирует, возвращает JSON

↓

DTO (Data Transfer Object) - объекты для API (Request/Response), отдельно от Entity, чтобы не светить внутреннюю структуру.

13. Краткая шпаргалка одним абзацем

Stylish - микросервисный гардероб: auth-service выдаёт JWT, wardrobe-service управляет профилем, вещами, фото и образами, generation-service генерирует картинку (пока stub). Postgres - основные данные через Flyway, Mongo - асинхронный аудит, MinIO - файлы. Spring Security + JWT HS256, refresh с ротацией. Frontend на React с авто-refresh токенов. Docker Compose поднимает 8 контейнеров. Swagger на каждом бэкенде. Cache (Caffeine) и Async для производительности.

Документ подготовлен для защиты проекта MUTANT4 / Stylish.